SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team: T-Junction

umhackathon@um.edu.my

Project: Zen Travel — Your AI Travel Concierge **Domain:** AI Systems & Agentic Workflow Automation
Version: 1.0 **Date:** 2026-04-26

---

1. Introduction

Purpose: This System Analysis Document captures the technical scope and design decisions behind Zen Travel — an agentic, multilingual travel concierge built on Z.AI's GLM as the central reasoning engine. Where the PRD explains what the product solves and for whom, this SAD explains how the product is engineered: the architecture, the data flows, the model orchestration boundaries, and the operational guard-rails that keep the agentic workflow reliable when external tools, networks, or the GLM itself misbehave.

Architecture: Zen Travel is a cloud-native web platform. The Next.js 15 App Router frontend talks to a thin Edge/API-route layer plus a FastAPI orchestration service that owns all GLM calls. Persistence and auth live in Supabase (PostgreSQL + Storage + Auth). External capabilities (maps, search, face liveness, weather, PDF generation) are isolated behind tool adapters so the GLM can call them through a single, schema-validated tool interface.

Data Flows: A user's natural-language intent — typed or spoken in any of 11 languages — flows from the browser to the API gateway, into the FastAPI workflow engine where the GLM produces a structured plan, then fans out to the right tool adapters (Maps, Search, MDAC PDF, Visa PDF, Weather, Supabase). Workflow state is checkpointed back into Supabase after every GLM step so that interrupted sessions can resume without losing the draft itinerary.

Model Process: The end-to-end agentic workflow covers the four planning stages a traveller actually goes through: (1) onboarding & profile capture, (2) plan-from-prompt itinerary generation, (3) live conversational concierge mid-trip, and (4) document automation (Visa eligibility check + Malaysia Digital Arrival Card pre-fill).

Role of Reference: This document is the reference for engineers building the system, QA engineers validating behaviour across the multi-step workflow, and judges evaluating how the GLM is wired in as an irreplaceable component rather than a generic AI module.

2. Background

Background: Travel planning today is fragmented across 10+ disconnected surfaces — flight aggregators, hotel sites, blogs, embassy portals, currency converters, review platforms, messaging apps. Each surface has its own

language assumptions, formats, and biases. Older travellers and non-English speakers are systematically excluded; younger travellers burn out from comparison fatigue. Group tours partially solved the effort problem but introduced rigid schedules, kickback shopping stops, and zero personalisation. The market gap is a system that absorbs vague, ambiguous traveller intent and returns a structured, budget-aware, personalised plan — without forcing the user to become a travel agent.

Previous Version: The codebase originated from the BizKuKu MSME template — a Next.js 15 scaffold targeted at small-business financial reporting and compliance workflows. The MSME-specific surfaces (financial reports, open finance, compliance dashboards) have been removed; the design tokens, internationalisation context, and Supabase auth wiring were retained and re-themed around the luxury-minimal palette (cream / charcoal / gold).

Changes in Major Architectural Components

- Replaced MSME finance microservices with travel-domain workflow orchestration in FastAPI (backend/main.py + backend/src/api/chat.py).
- Introduced the GLM service layer as a first-class component (Z.AI ILMU GLM 5.1) with schema-validated tool calls.
- Added agent-tool adapters under src/app/api/* (chatbot, itinerary, mdac-pdf, visa-pdf, recommendations, weather, conflict-news, voice-prefs, face-recognition).
- Re-platformed authentication around AWS Rekognition + MediaPipe Tasks Vision face-liveness in front of Supabase Auth.
- Switched the persistence model from finance ledgers to a traveller-centric schema: profiles, itineraries, itinerary_days, activities, documents, sessions.

New Capabilities Introduced in This Version

- GLM-driven multi-step itinerary synthesis with map + draggable timeline rendering.
- Multilingual concierge (11 languages: EN, BM, ZH, TA, JA, KO, AR, FR, ES, DE, HI) with text and voice input.
- Visa requirement lookup keyed by nationality + destination, plus pre-filled Malaysia Digital Arrival Card (MDAC) PDF export.
- Face-liveness gated authentication binding the booked documents to a real, present human.
- Stateful workflow checkpointing — every GLM step writes back to Supabase so interrupted sessions resume cleanly.
- Schema-validated tool layer with retry, grounding against Maps/Search, and graceful fallback to a partial result + human-handoff form.

Target Stakeholders

Stakeholder roles and what each party expects from Zen Travel during the hackathon delivery and beyond.

Stakeholder	Role	Expectations
Traveller (End User)	Describes intent in natural language, reviews the AI-generated itinerary, edits via the concierge, downloads MDAC + visa documents.	A calm, single-surface planning experience in their native language; itineraries that respect budget and pace; documents that work at the border.

Concierge (AI Agent)	GLM-driven orchestrator that decomposes intent, calls tools, and returns structured results.	Reliable schema validation, deterministic tool routing, clear failure semantics so the agent never silently hallucinates.
Travel Operations (Future)	Human escalation for rejected workflows (e.g. visa denial, sold-out stays, payment issues post-MVP).	Full conversation transcript + workflow state hand-off, structured incident metadata.
Development Team	Builds and maintains the Next.js frontend, FastAPI orchestrator, tool adapters, and Supabase schema.	Clear API contracts, modular tool adapters, schema-versioned GLM prompts, observable workflow state.
QA Team	Validates the agentic workflow across all 11 languages and all four sub-workflows including failure paths.	Per-language eval set, deterministic replay of GLM transcripts, defined edge cases (ambiguity, missing data, tool outage).
Hackathon Judges	Evaluate the system against Domain 1 expectations: multi-step orchestration, clear architecture, edge-case handling.	Evidence that removing the GLM collapses the product; demonstrable failure recovery; clean architectural diagrams.

3. System Architecture & Design

3.1 High Level Architecture

Overview: Zen Travel is a client–server, cloud-native system with three logical tiers. (1) The Client tier is a responsive Next.js 15 web application with React 19, Tailwind, Leaflet maps, and a floating concierge component. (2) The Application tier splits into a Next.js Edge/API-route layer for thin, stateless tool adapters (PDF generation, weather lookup, recommendations, face-recognition session) and a Python FastAPI orchestration service that owns every GLM call, state checkpoint, and multi-step plan execution. (3) The Data tier is Supabase, providing PostgreSQL persistence, object storage (passport scans, generated PDFs), and JWT auth. External services — Z.AI GLM, AWS Rekognition, mapping & search providers — are reached through dedicated adapters so they can be swapped or rate-limited without touching the orchestrator core.

System & Architecture Snapshot

Type	Details
System	Responsive web application (desktop + mobile browsers) — Next.js 15 App Router, React 19, TypeScript.
Architecture	Cloud-native, three-tier client–server with an agentic workflow service. GLM-as-Service layer decoupled behind a schema-validated tool bus.
Auth	Face-liveness (AWS Rekognition + MediaPipe Tasks Vision) gating Supabase Auth JWT.
Persistence	Supabase PostgreSQL (3NF schema) + Supabase Storage for binary artefacts (PDFs, scans).
Deployment Target	Vercel (Next.js frontend + Edge API routes), Fly.io / Render (FastAPI orchestrator), Supabase managed cloud, AWS region for Rekognition.
Observability	Structured JSON logs from FastAPI; per-step GLM trace IDs persisted to a workflow_runs table for replay & debugging.

3.2 LLM as Service Layer (GLM Integration)

The architecture treats Z.AI's GLM as a first-class service layer, not a bolted-on "AI module". Every GLM call is mediated by a thin client in the FastAPI orchestrator that handles prompt assembly, context shaping, schema validation, retry, and grounding. If the GLM service layer is removed, the orchestrator can no longer classify intent, plan the workflow, or coordinate the tool fan-out — the product collapses.

Dependency Diagram (Logical View)

```
+-----------------------------------------------------------------+
|                          CLIENT TIER                            |
|  +--------------------+   +--------------------+                |
|  | Next.js 15 (React) |   | Floating Concierge |  (text + voice) |
|  | Pages: /home,      |<->| Chatbot.tsx +      |  i18n: 11 langs |
|  | /onboarding,       |   | ReasoningPanel.tsx |                |
|  | /itinerary, /login |   +--------------------+                |
|  +---------+----------+             |                           |
+------------|---------------------------|---------------------------+
             |  HTTPS / fetch            |  HTTPS / SSE
             v                           v
+-----------------------------------------------------------------+
|                  EDGE / API-ROUTE LAYER (Next.js)               |
|   /api/chatbot   /api/itinerary   /api/mdac-pdf   /api/visa-pdf  |
|   /api/recommendations   /api/weather   /api/conflict-news       |
|   /api/voice-prefs   /api/face-recognition                       |
+--------------------------------+--------------------------------+
                                 |  REST (JSON, schema-validated)
                                 v
+-----------------------------------------------------------------+
|              FASTAPI ORCHESTRATOR (workflow engine)              |
|                                                                  |
|   +---------------------+        +-------------------------+     |
|   | Intent Router       |  --->  | Workflow Planner (GLM)  |     |
|   | (zero-shot classify)|        | emits JSON plan         |     |
|   +---------------------+        +-----------+-------------+     |
|                                              |                   |
|                                              v                   |
|   +-------------------------------------------------------+       |
|   |      GLM SERVICE LAYER  (Z.AI ILMU GLM 5.1)          |       |
|   |  - Prompt builder (role-scoped sub-prompts)          |       |
|   |  - Context window manager (chunk + summarise)        |       |
|   |  - Schema validator + 1-shot retry                   |       |
|   |  - Grounding check vs. Maps/Search                   |       |
|   |  - Cost router (small-tier for trivial follow-ups)   |       |
|   +-----+-----------+-----------+-----------+------------+       |
|         |           |           |           |                    |
|         v           v           v           v                    |
|   +---------+  +---------+  +---------+  +---------+             |
|   | Profiler|  |Itinerary|  |  Visa   |  |Concierge|   ...       |
|   |  prompt |  | Builder |  | Advisor |  |  prompt |             |
|   +----+----+  +----+----+  +----+----+  +----+----+             |
|        |           |           |           |                     |
|        +-----+------+------+-----+------+-----+                  |
|              |            |           |                          |
|              v            v           v                          |
|        TOOL ADAPTER BUS  (schema-validated function calls)       |
|        Maps | Search | PDF (pdf-lib) | Weather | Supabase        |
+--------|------|------|------|--------|--------|------------------+
         v      v      v      v        v        v
     +-------+-------+-------+--------+--------+--------+
     | Z.AI  | AWS   |Mapping| Open   | Supabase (PG +  |
     | GLM   | Rekog | API   | Search | Storage + Auth) |
     +-------+-------+-------+--------+-----------------+
```

Prompt Construction & Context Window Management

- **Prompt construction:** A top-level Planner prompt receives the user's raw turn plus a compact rolling profile (preferences, budget, locale). It returns a strict JSON workflow plan: an ordered list of (step_name, required_tools, expected_output_schema) tuples.
- **Context window:** each sub-step gets only the slice of state it needs — Profiler sees onboarding answers; Itinerary-Builder sees profile + destination + budget; MDAC-Filler sees passport OCR + nationality. We never ship the full conversation into every step; this keeps token cost predictable.
- **Token enforcement:** user input is capped at 8,000 characters per turn. Conversation history is summarised into a rolling profile object once it exceeds 4,000 tokens. Uploaded documents are OCR'd, chunked at 1,500 tokens with 150-token overlap, embedded; only the top-3 relevant chunks reach the prompt.
- **Response parsing:** each GLM call declares an expected JSON schema. Responses are validated; on failure we retry once with a stricter "return JSON only" reminder; a second failure surfaces a partial result with a human-readable explanation rather than crashing the workflow.
- **Hand-off to next component:** the validated, parsed object is forwarded to the next step's tool adapter (e.g. Itinerary-Builder output is passed to the Maps adapter for grounding before it reaches the UI).

Major API Calls Between Components

From	To	Purpose	Transport
Browser	Next.js /api/chatbot	Submit a user concierge turn (text or transcribed voice).	HTTPS POST (JSON)
Next.js /api/chatbot	FastAPI /v2/chat	Forward the turn into the orchestrator with session_id and locale.	HTTPS POST (JSON)
FastAPI orchestrator	Z.AI GLM API	Send the assembled prompt + tool schemas; receive the validated plan or step output.	HTTPS POST (JSON / function-calling)
FastAPI orchestrator	Maps / Search adapter	Ground itinerary entities, fetch coordinates and opening hours.	HTTPS REST
FastAPI orchestrator	Supabase REST	Read/write profile, itinerary, itinerary_days, activities, sessions, documents.	HTTPS REST (PostgREST)
Browser	Next.js /api/mdac-pdf	Generate a pre-filled MDAC PDF for the current trip.	HTTPS POST (JSON) → application/pdf
Browser	Next.js /api/visa-pdf	Export the visa-eligibility decision document.	HTTPS POST (JSON) → application/pdf
Browser	Next.js /api/face-recognition	Verify face-liveness session + match against the stored embedding.	HTTPS POST (multipart)
Browser	AWS Rekognition	Performs the actual liveness session via the AWS SDK on the client (signed via API).	HTTPS (AWS SDK)
Browser	Next.js /api/weather, /api/conflict-news, /api/recommendations	Fetch contextual data the UI overlays on the itinerary.	HTTPS GET (JSON)

3.3 Sequence Diagram — Plan-from-Prompt User Journey

Walk-through of the highest-value user flow: a logged-in traveller types "5 days in Penang under RM2,500, calm pace, vegetarian" into the concierge and receives an interactive itinerary on the map + timeline.

```
Traveller    Browser     Next.js       FastAPI         GLM         Tool         Supabase
  (UI)       (Client)    (/api/*)     (Orchestrator)  (Z.AI)      Adapters      (DB)
   |            |           |              |             |            |             |
   | type req   |           |              |             |            |             |
   |----------->|           |              |             |            |             |
   |            | POST chat |              |             |            |             |
   |            |---------->|              |             |            |             |
   |            |           | POST /v2/chat|             |            |             |
   |            |           |------------->|             |            |             |
   |            |           |              | load profile|            |             |
   |            |           |              |------------------------->|             |
   |            |           |              |<-------------------------|             |
   |            |           |              | Planner prompt           |             |
   |            |           |              |------------>|            |             |
   |            |           |              |<------------|            |             |
   |            |           |              | (workflow plan JSON)     |             |
   |            |           |              | step: search anchors     |             |
   |            |           |              |--------------------->|   |             |
   |            |           |              |<---------------------|   |             |
   |            |           |              | Itinerary-Builder prompt|            |
   |            |           |              |------------>|            |             |
   |            |           |              |<------------|            |             |
   |            |           |              | ground vs. Maps          |             |
   |            |           |              |--------------------->|   |             |
   |            |           |              |<---------------------|   |             |
   |            |           |              | persist itinerary        |             |
   |            |           |              |------------------------->|             |
   |            |           |              | checkpoint workflow_run  |             |
   |            |           |              |------------------------->|             |
   |            |           |  itinerary   |                          |             |
   |            |           |<-------------|                          |             |
   |            |  render   |              |                          |             |
   |            |<----------|              |                          |             |
   | see plan   |           |              |                          |             |
   |<-----------|           |              |                          |             |
```

Key behaviours visible in the diagram:

- Two distinct GLM calls: Planner (workflow plan) and Itinerary-Builder (per-day breakdown). Each is schema-validated independently.
- Tool adapters run between GLM calls — Search to seed candidate anchors, Maps to ground generated entities against real coordinates.
- A workflow_runs row is checkpointed before the response returns, so a network drop after this point does not lose the draft.
- If the second GLM call returns malformed JSON, the orchestrator retries once with a stricter prompt; if it fails again, a partial itinerary is rendered with the failed days marked for human review.

3.4 Technological Stack

Layer	Technology	Rationale
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS 4, Framer Motion, Leaflet + react-leaflet	App Router gives us co-located server actions for tool adapters; React 19 + Framer Motion power the concierge animation; Leaflet renders the itinerary map without paid Maps quota.

UI System	Tailwind tokens (cream/charcoal/gold), Playfair Display (serif headings) + Inter (body), lucide-react icons, react-markdown for GLM-rendered output	Luxury-minimal aesthetic differentiates the concierge from generic travel chatbots.
Backend Orchestrator	FastAPI (Python 3.10+), Pydantic schemas, Uvicorn	Pydantic gives first-class JSON-schema validation for every GLM call; FastAPI's async model fits the tool fan-out pattern.
AI / Reasoning	Z.AI ILMU GLM 5.1 (primary), Google Generative AI (gemini-1.5-flash) as a development-time fallback in backend/main.py	GLM is the irreplaceable reasoning engine; the Gemini path stays only as a local-dev fallback when the GLM key is unset.
Auth & Identity	Supabase Auth (JWT), AWS Rekognition face-liveness, @mediapipe/tasks-vision for client-side face detection	Face-liveness binds documents to a present human; Supabase Auth handles session JWT and row-level security.
Database	Supabase PostgreSQL (3NF schema), Supabase Storage for PDF and scan blobs	Single managed Postgres + Storage backend keeps operations small for hackathon scope while supporting RLS.
PDF Generation	pdf-lib (TypeScript) running inside Next.js Edge/Node API routes	MDAC and Visa export PDFs are generated deterministically without an extra service.
External Data	Mapping/search provider, OpenWeather-style weather, conflict/news feed	Wrapped behind tool adapters so a single provider swap is a one-file change.
Cloud / Deployment	Vercel (frontend + Edge API routes), Fly.io or Render (FastAPI), Supabase managed, AWS for Rekognition	Free/low-cost tiers across Vercel + Supabase keep the hackathon footprint small while leaving headroom for scale.
Observability	Structured JSON logs (FastAPI), workflow_runs audit table, Vercel & Supabase native dashboards	Every GLM step is replayable from the workflow_runs row — critical for debugging agentic failures.

4. Key Data Flows

4.1 Data Flow Diagram (DFD — Level 1)

Movement of data across users, processes, external services, and datastores. No "black holes" (data sinks with no consumer) or "miracles" (data appearing without a source) — every store has a writer and a reader.

```
[Traveller] -- intent (text/voice, 11 langs) --> ( P1: Capture & Translate )
                                                          |
                                                normalised intent
                                                          v
[Traveller] -- passport scan ----------------> ( P2: Document Ingest )
                                                  |        |
                                        OCR text  |        |  raw scan
                                                  v        v
                                          ( P3: Workflow Planner / GLM )
                                                  |  ^
                                        step plan |  | profile, history
                                                  v  |
( P4a: Itinerary  )  ( P4b: Visa  )  ( P4c: MDAC  )  ( P4d: Concierge )
     |   ^               |   ^            |   ^             |   ^
     |   |               |   |            |   |             |   |
```

```
        v   |              v   |           v   |            v   |
   +----------+        +----------+      +----------+      +----------+
   | Maps API |        | Visa     |      | Form     |      | Search/  |
   | (extern) |        | rules KB |      | renderer |      | Weather  |
   +----------+        | (extern) |      | (pdf-lib)|      | (extern) |
                       +----------+      +----------+      +----------+
        |                  |                 |                 |
        +---------+---------+--------+--------+---------+-------+
                  |                  |                  |
                  v                  v                  v
             ( P5: Persist & Checkpoint  -- writes workflow_runs )
                  |
   +--------------+---------------+-----------------+
   |              |               |                 |
   v              v               v                 v
[ D1: profiles ][ D2: itineraries ][ D3: documents ][ D4: workflow_runs ]
                                                      |
                                         periodic export   v
                                                   ( P6: Telemetry / Replay )
                                                         |
                                                         v
                                                     [ Dev / QA ]
```

DFD law compliance:

- No black holes: every datastore (D1–D4) has a downstream reader (P4a–d, P6, or the Browser via Supabase REST).
- No miracles: every output is traceable to an input — itinerary entities only enter D2 after grounding by P4a.
- No grey holes: process P3 (Workflow Planner / GLM) cannot produce a plan without both the normalised intent and the loaded profile.

4.2 Normalised Database Schema (3NF)

Logical data model in Third Normal Form. Junction tables resolve the Many-to-Many relationships (e.g. activities ↔ tags). Foreign keys cascade on user deletion to honour data-purge requests.

```
+----------------+        +----------------+        +-----------------+
|    users       |1     1 |    profiles    |1     N |  itineraries    |
|----------------|--------|----------------|--------|-----------------|
| id (PK)        |        | user_id (PK,FK)|        | id (PK)         |
| email (UQ)     |        | display_name   |        | user_id (FK)    |
| auth_provider  |        | locale         |        | title           |
| face_embed_url |        | budget_band    |        | destination     |
| created_at     |        | pace           |        | start_date      |
+----------------+        | dietary        |        | end_date        |
                          | accessibility  |        | budget_total    |
                          +----------------+        | status          |
                                                    | created_at      |
                                                    +--------+--------+
                                                             |
                                                            1| N
                                                             v
                                                    +-----------------+
                                                    | itinerary_days  |
                                                    |-----------------|
```

```
                                                  | id (PK)         |
                                                  | itinerary_id FK |
                                                  | day_number      |
                                                  | date            |
                                                  | summary         |
                                                  +-------+---------+
                                                          | 1
                                                          | N
                                                          v
                                                  +-----------------+
                                                  |   activities    |
                                                  |-----------------|
                                                  | id (PK)         |
                                                  | day_id (FK)     |
                                                  | slot (am/pm/eve)|
                                                  | title           |
                                                  | place_id        |   <--- grounded by Maps adapter
                                                  | lat, lng        |
                                                  | est_cost        |
                                                  | verified (bool) |
                                                  +-------+---------+
                                                          | N
                                                          |
                                                          v
                           +----------------+        +-----------------+
                           |     tags       |1     N | activity_tags   |  (junction; resolves M:N)
                           |----------------|--------|-----------------|
                           | id (PK)        |        | activity_id FK  |
                           | name (UQ)      |        | tag_id      FK  |
                           +----------------+        +-----------------+

+----------------+        +-----------------+        +-----------------+
|   documents    |N     1 |     users       |        | workflow_runs   |
|----------------|--------|-----------------|        |-----------------|
| id (PK)        |        | (see above)     |        | id (PK)         |
| user_id (FK)   |        +-----------------+        | user_id (FK)    |
| kind (mdac/    |                                   | itinerary_id FK |
|       visa/    |                                   | step_name       |
|       passport)|                                   | input_hash      |
| storage_path   |                                   | output_json     |
| issued_at      |                                   | status          |
| expires_at     |                                   | latency_ms      |
+----------------+                                   | tokens_in/out   |
                                                     | created_at      |
                                                     +-----------------+

+----------------+        +-----------------+
|   sessions     |N     1 |     users       |
|----------------|--------|-----------------|
| id (PK)        |        +-----------------+
| user_id (FK)   |
| device         |
| locale         |
| started_at     |
| last_seen_at   |
+----------------+
```

Normalisation notes:

- activity_tags is a junction table that resolves the Many-to-Many between activities and tags — without it, tag names would repeat per activity, violating 2NF.
- profiles is split out from users so authentication concerns (email, provider, embedding URL) do not entangle with travel-preference attributes that change frequently.
- workflow_runs is append-only and serves as the audit log for every GLM step — input_hash + output_json make every run replayable for QA.
- Storage paths in documents point to Supabase Storage; rows never embed binary blobs, so query plans stay small.

5. Functional Requirements & Scope

5.1 Minimum Viable Product (MVP)

Core features Team T-Junction will demo at the end of UMHackathon 2026. Each item is end-to-end working — UI, GLM call, tool adapter, persistence — not a stub.

#	Feature	Description
1	Face-Liveness Login & Onboarding	User signs in via AWS Rekognition + MediaPipe face-liveness, then completes a 4-step onboarding (persona, budget band, pace, dietary/accessibility). The GLM compiles the answers into a structured profile in Supabase.
2	Plan-from-Prompt Itinerary Generation	User describes the trip in any of 11 languages. The GLM emits a workflow plan, calls the Maps + Search adapters, and returns a day-by-day itinerary rendered on a Leaflet map and draggable timeline.
3	Multilingual AI Concierge (Text + Voice)	A floating concierge accepts text or voice input, answers in the user's locale, and can mutate the itinerary in place ("swap day 2 for something quieter") while preserving the rest of the plan.
4	Visa Eligibility + MDAC Pre-fill PDF	User uploads a passport scan. GLM extracts nationality via OCR, looks up visa rules per nationality+destination, drafts the Malaysia Digital Arrival Card payload, and exports a pre-filled PDF via pdf-lib.
5	Stateful Workflow Recovery	Every GLM call writes a workflow_runs checkpoint. If a tool call fails or the user closes the tab mid-plan, the next session resumes from the last checkpoint instead of starting over.

5.2 Non-Functional Requirements (NFRs)

System qualities that are not features but are critical for a judge-grade demo and any post-hackathon scale-up.

Quality	Requirement	Implementation
Reasoning Reliability	≥ 95% of GLM calls return a payload that passes its declared JSON schema on first or second attempt.	Pydantic-validated schemas per step; one strict-mode retry; failure path returns a partial result + human-readable explanation rather than a crash.
Token Latency	p95 GLM round-trip ≤ 3s under normal load; full itinerary generation ≤ 12s warm, ≤ 25s cold.	5s async timeout per call; loading skeleton + step-by-step "reasoning panel" surfaced in the UI; tool calls fan out in parallel where independent.
Cost Efficiency	Average GLM tokens per completed itinerary stay under the team-defined budget; tokens-per-itinerary is the north-star efficiency metric.	Fixed-length system prompts; user input truncated/chunked at 8,000 chars; deterministic sub-prompts (visa rules, language scaffolds) cached for 24h keyed by hash; small-tier GLM for trivial follow-ups.

Multilingual Quality	Itinerary quality must not visibly degrade across the 11 supported languages.	Per-language eval set in QA; fallback path drafts in English then translates if a language-specific quality check fails.
Reliability	Itinerary draft is never lost on a single component failure (browser crash, tool 500, GLM rate-limit).	workflow_runs checkpoint after every GLM step; sessions table tracks last_seen_at for resume; Supabase row-level security prevents cross-user leakage.
Scalability	Burst traffic (e.g. demo day) up to 100 concurrent active sessions without queueing user-visible turns > 5s.	Stateless FastAPI horizontal scale; Supabase connection pool; GLM concurrency capped per region with a backpressure queue and friendly "busy" UI state.
Maintainability	Each tool adapter (Maps, Search, PDF, Weather) can be swapped with a one-file change.	Adapter interface + JSON schema per tool; CI runs schema-contract tests against recorded fixtures.
Security & Privacy	Passport scans and MDAC payloads encrypted at rest; auto-purged 30 days post-trip; opt-in retention only.	Supabase Storage with signed URLs; RLS policies per user_id; scheduled purge job on documents.expires_at; explicit consent toggle in onboarding.
Accessibility	WCAG AA targets; voice input + readable contrast on cream/charcoal palette; full keyboard navigation.	Tailwind tokens audited for contrast; voice transcription via Web Speech API; ARIA labels on the concierge launcher and timeline drag handles.
Observability	Every agentic failure is reproducible from logs alone within 5 minutes.	workflow_runs row stores input_hash, output_json, latency_ms, tokens_in/out, status; structured JSON logs; replay tooling that re-issues the recorded prompt.

5.3 Out of Scope / Future Enhancements

Features the team has scoped out of the UMHackathon 2026 deliverable but earmarked for the post-hackathon "Grand Vision":

- Live booking via real GDS / OTA inventory (flights, hotels, activities) — payment is mocked in the MVP.
- Group / multi-user simultaneous editing of the same itinerary with presence indicators.
- Native mobile apps (iOS / Android) — the hackathon ships a responsive web app only.
- Offline-first mode with locally cached itineraries and deferred GLM sync on reconnect.
- Insurance brokering, currency hedging, and post-trip expense reconciliation.
- Pre-trip community features (shared itinerary forking, traveller reviews of AI-generated plans).
- Voice-first phone IVR for travellers without a smartphone.

6. Monitor, Evaluation, Assumptions & Dependencies

6.1 Technical Evaluation

Grayscale Rollout & A/B Testing: New GLM prompts and tool adapters are released behind a feature flag to 5% of real users first. We watch the workflow_runs success rate, p95 latency, and tokens-per-itinerary; if all three stay within tolerance for 24h, the flag widens to 25% → 50% → 100%. For the Itinerary-Builder prompt specifically, we run an A/B between the current variant and a candidate with stricter few-shot examples; the winner is decided on a composite of grounding-pass rate and average user-edit count after generation.

Emergency Rollback (ER) & Golden Release: The last fully-validated build is tagged as the Golden Release in Vercel + Fly.io. If the Priority Matrix triggers (see below), the automated pipeline redeploys the Golden tag and disables the offending feature flag. Database migrations are forward-compatible (additive columns, no destructive renames) so a rollback never requires a schema downgrade. Rollback target: under 5 minutes from alert to Golden serving.

6.2 Priority Matrix

Priority	Trigger Condition	Action
P1 — Critical	GLM schema-validation failure rate > 5% over 5 minutes, or face-liveness service down, or > 1% workflow_runs left in `errored` state.	PagerDuty page; emergency rollback to Golden Release; freeze new sessions on the affected sub-workflow with a "we're catching up" UI state.
P2 — High	p95 itinerary latency > 25s for 10 consecutive minutes, or tokens-per-itinerary > 2× baseline.	Disable the offending feature flag; pin to small-tier GLM for low-complexity follow-ups; investigate within the active sprint.
P3 — Medium	A single language's quality eval set fails > 10% (e.g. Tamil itineraries fall below threshold).	Activate English-draft-then-translate fallback for that language; queue a prompt-tuning task in the next sprint.
P4 — Low	Cosmetic UI regression (e.g. timeline drag handle misaligned on Safari).	File issue, fix in the next regular release.

6.3 Monitoring

We monitor four signals continuously, each with a defined alert threshold:

- **Reasoning health:** GLM call status, latency, tokens-in/out, schema-validation pass/fail. Persisted to workflow_runs and exposed on a dashboard. Alerts: schema-fail > 5% / 5min, p95 > 5s / 5min.
- **API health:** request rate, p95 latency, error rate per /api route. Alerts: 5xx rate > 1% / 5min.
- **Persistence health:** Postgres connection pool utilisation, Storage egress per user, RLS policy violation count (must remain zero). Alerts: pool > 80%, RLS violation > 0.
- **User-journey health:** face-liveness completion rate, drop-off per onboarding step, share of itineraries the user edits within 5 minutes (proxy for AI quality). Alerts: liveness pass < 90%, onboarding drop-off > 30% per step.

6.4 Assumptions

- Travellers have a stable internet connection during planning and during MDAC export — the agentic workflow assumes synchronous network access to GLM and tool adapters.
- Travellers will agree to a brief face-liveness step at signup; users on devices without a working camera fall back to email/password (post-MVP).
- The GLM provider's rate limits and uptime SLA hold during the demo window; otherwise the small-tier router and graceful fallback take over.
- Passport scans uploaded by users are genuine; we do not perform document forensics in the MVP, only OCR + nationality lookup.
- The hackathon judging environment supports a modern browser (Chrome/Edge/Safari latest) — our face-liveness pipeline depends on getUserMedia + WebAssembly for MediaPipe.

- The Supabase free tier suffices for demo-scale traffic (≤ 100 concurrent sessions); production scale-up would move to a paid tier.

6.5 External Dependencies

Tool / Service	Purpose	Risk & Mitigation
Z.AI ILMU GLM 5.1 (API)	The core LLM reasoning engine — handles intent classification, multi-step planning, structured output generation, and tool-call selection.	HIGH. If the GLM API is rate-limited or down, the product collapses. Mitigations: graceful degradation UI ("the concierge is resting"), small-tier router for trivial follow-ups, 24h cache for deterministic sub-prompts (visa rules, language scaffolds), workflow checkpoint so the user does not lose draft state.
AWS Rekognition + MediaPipe Tasks Vision	Face-liveness verification at login and account binding.	MEDIUM. If Rekognition is unavailable, login is blocked. Mitigation: feature-flag fallback to Supabase password auth for the demo; client-side MediaPipe still gives a basic liveness signal even if Rekognition matching fails.
Supabase (PostgreSQL + Storage + Auth)	Persistence (profiles, itineraries, documents, workflow_runs), object storage (PDFs, scans), and JWT auth.	MEDIUM. Single managed dependency; an outage halts persistence. Mitigation: read-only mode that lets the active session continue against in-memory state; Supabase has multi-region failover on paid tiers (post-hackathon).
Mapping / Search Provider	Grounds GLM-generated entities (attractions, hotels) against real coordinates and opening hours.	MEDIUM. If the maps provider returns errors, generated itineraries cannot be grounded. Mitigation: unverified entities are flagged in the UI with a yellow badge instead of being silently shown; user can manually approve.
pdf-lib (in-process)	MDAC and Visa PDF generation inside Next.js API routes.	LOW. In-process library; failure mode is a single failed export, not a system-wide outage. Mitigation: retry on the client; export-text fallback so the user can copy the payload manually.
Weather / Conflict-News providers	Contextual data overlaid on the itinerary (weather warnings, travel advisories).	LOW. Overlay-only; an outage degrades context but does not block planning. Mitigation: cache last-known values for 6h; UI shows "data unavailable" rather than failing.
Vercel & Fly.io / Render	Hosting for Next.js (frontend + Edge API routes) and FastAPI orchestrator respectively.	LOW. Standard PaaS providers with strong uptime. Mitigation: Golden Release tag pinned for instant rollback.

7. Project Management & Team Contributions

Preliminary round = ~10 days of build inside a two-week sprint. The plan is structured to land a working end-to-end demo by Day 7 so the remaining days can be spent on evaluation, polish, and edge-case hardening rather than firefighting.

7.1 Project Timeline

Day	Phase	Focus
Day 1–3	Project Planning & Foundations	Finalise PRD/SAD, lock the GLM service-layer contract (prompt + schema per step), set up Supabase schema + RLS, scaffold the FastAPI orchestrator and tool-adapter interface.

Day 3–6	Core Workflow — Itinerary	Implement Planner + Itinerary-Builder GLM steps, Maps + Search adapters, the timeline + Leaflet map UI. End-of-Day-4 milestone: a real prompt produces a real, grounded itinerary persisted in Supabase.
Day 6-8	Multilingual Concierge	Wire the floating concierge to the orchestrator, add the 11-language i18n scaffold, voice input via Web Speech API, in-place itinerary mutation flow.
Day 6	Documents — Visa + MDAC	Passport OCR adapter, visa-rules lookup, MDAC payload generator, pdf-lib export route. End-of-Day-6 milestone: a downloadable pre-filled MDAC PDF for a demo nationality.
Day 7	Auth & Resume	Face-liveness flow (Rekognition + MediaPipe), workflow_runs checkpoint logic end-to-end, session resume UX. End-of-Day-7 milestone: full happy path demo-ready.
Day 8	Evaluation & Edge Cases	Per-language eval set, chaos test (kill maps, rate-limit GLM, malformed JSON), Priority Matrix triggers verified, monitoring dashboard wired.
Day 9	Polish & Performance	Reduce p95 latency, prompt-cost tuning (token cap, sub-prompt caching), accessibility pass, copy review across all 11 languages.
Day 10	Demo Prep & Submission	Final demo script + recorded video, judge walkthrough rehearsal, final SAD/PRD lock, repository hygiene (README, env example, run scripts).

7.2 Team Members & Roles — Team T-Junction

Members	Role	Responsibility
Tiong Tsui Jeff	AI / Orchestration Lead	Owns the GLM service layer: prompt design (Planner, Itinerary-Builder, Visa-Advisor, MDAC-Filler, Concierge), JSON schemas, retry/grounding logic, cost-router, evaluation harness.
Teh Jia Jie	Backend Lead	FastAPI orchestrator, tool-adapter bus, Supabase schema + RLS policies, workflow_runs checkpointing, observability (structured logs + replay tooling).
Tan Hong Han	Frontend Lead	Next.js App Router pages (login, onboarding, home, itinerary, trip-setup, explore), Leaflet map + draggable timeline, floating concierge, ReasoningPanel, i18n integration across 11 languages.
Tay Zheheng	Auth & Identity Engineer	AWS Rekognition + MediaPipe face-liveness pipeline, Supabase Auth wiring, FaceLivenessModal component, signed-URL document storage.
Boon Mi Suet	QA & Evaluation Lead	Per-language eval set, chaos testing (tool outages, malformed GLM JSON, rate limits), Priority Matrix verification, demo-day rehearsal scripting.

Note: Team T-Junction is operating in a small-team format; members carry adjacent responsibilities (e.g. Frontend Lead also owns accessibility; Backend Lead pairs on observability with QA). Specific individual assignments are tracked in the team's project board.

7.3 Recommendations

- Cache deterministic sub-prompts (visa rules per nationality+destination, language-specific system scaffolds) in Redis or Supabase keyed by SHA-256 of the inputs — measured token saving in early tests is meaningful enough to justify the operational overhead.
- Introduce a small-tier GLM router for trivial follow-ups ("what's the weather in Penang day 3") to cap average tokens-per-session; reserve full-tier for Planner and Itinerary-Builder steps.
- Add a tokens-per-completed-itinerary north-star metric to the dashboard from Day 1 — agentic systems are easy to bloat, hard to trim retroactively.

- Keep all GLM prompts versioned in source control with a prompt_version field on workflow_runs; this turns prompt regressions into a normal git-bisect problem instead of a forensics exercise.
- Run chaos tests against the tool layer at least once per sprint (kill the maps adapter, force malformed JSON, throttle GLM) — the agentic workflow's value proposition is robustness, so it must be exercised, not asserted.
- Post-hackathon, evaluate moving the FastAPI orchestrator behind a queue (Cloud Tasks / SQS) so long-running multi-step plans survive request timeouts without sacrificing the streaming UI.